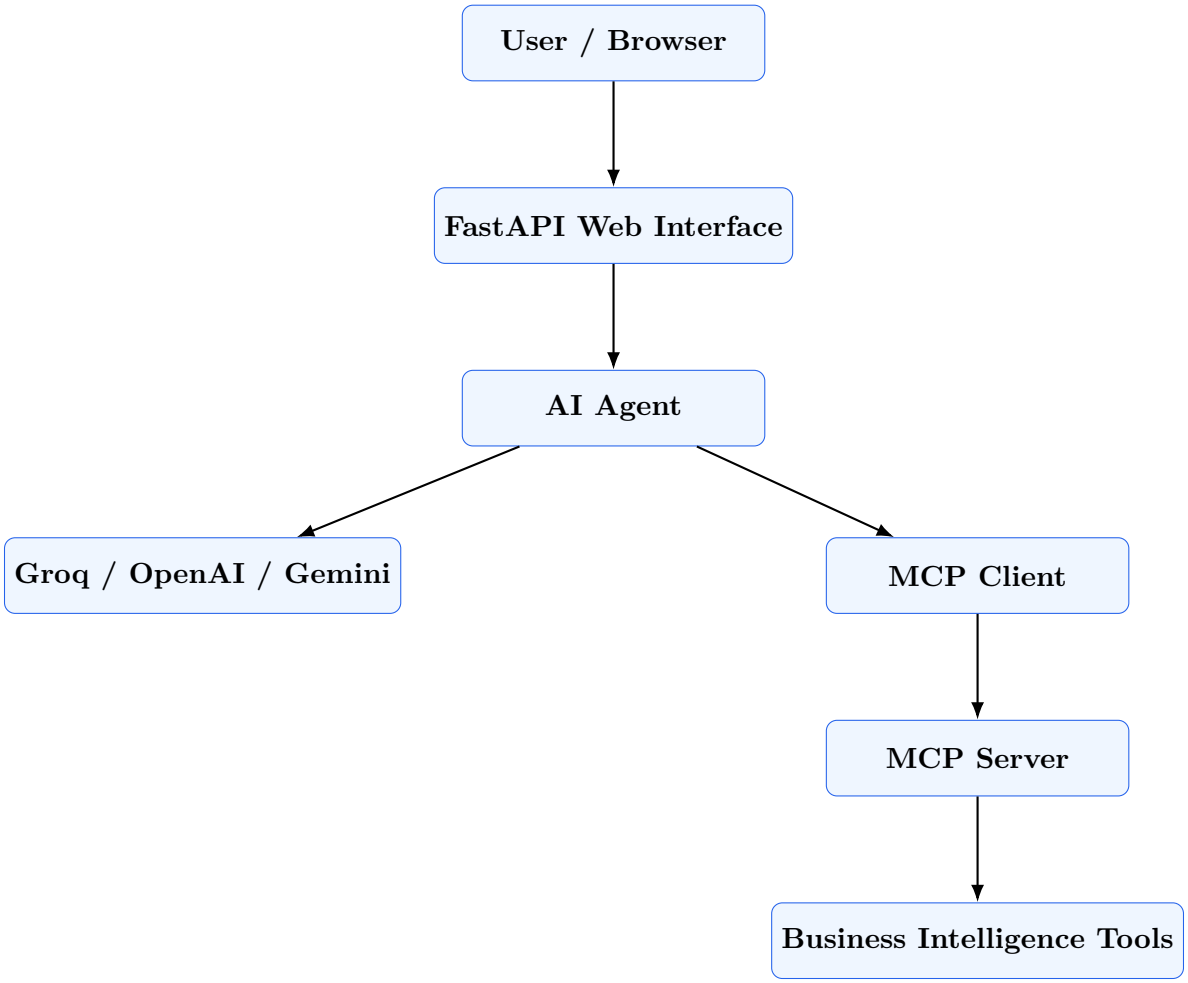
MCP Business Intelligence Agent

Model Context Protocol Server, Client, and AI Agent



Technical Documentation

MCP / LLM / FastAPI / Python

September 1, 2026

Contents

1 Project Overview

1.1 Purpose

The MCP Business Intelligence Agent is an AI-powered application that demonstrates how the Model Context Protocol (MCP) can be used to connect large language models to external business intelligence tools in a controlled and standardized manner.

The application consists of four major layers:

1. A web interface through which the user asks business questions.
2. An AI agent responsible for reasoning about the user's request.
3. An MCP client that communicates with the MCP server.
4. An MCP server that exposes business intelligence tools.

The system is designed to support multiple LLM providers, including Groq, OpenAI, and Google Gemini.

The key architectural principle is that the LLM does not directly access the underlying business system. Instead, it requests an appropriate tool through the MCP client, and the MCP server executes the permitted operation.

1.2 Example Use Case

A user can ask:

> "Which product is underperforming and what is its growth rate?"

The AI agent determines that business data is required, selects the appropriate MCP tool, invokes it through the MCP client, receives the result from the MCP server, and finally asks the LLM to formulate a human-readable response.

2 Learning Objectives

The project demonstrates the following concepts:

- Model Context Protocol (MCP)
- MCP server implementation
- MCP client implementation
- Streamable HTTP communication
- LLM tool/function calling
- AI agent architecture
- FastAPI application development
- Environment-based secret management
- Multi-provider LLM architecture
- Business intelligence automation
- Local development and testing
- Cloud deployment

3 Technology Stack

Technology	Purpose
Python	Core programming language
FastAPI	Web application and HTTP API
Uvicorn	ASGI server
MCP Python SDK	MCP server/client implementation
Groq SDK	Groq LLM integration
OpenAI SDK	OpenAI integration
Google Gemini SDK	Gemini integration
python-dotenv	Environment variable management
HTML/CSS/JavaScript	Web interface
Git/GitHub	Version control and repository hosting

Table 1: Technology stack

4 Repository Architecture

A recommended repository structure is:

```
mcp-business-intelligence/
|
+-- agent/
| +-- init.py
| +-- agent.py
| +-- test_agent.py
|
+-- client/
| +-- init.py
| +-- client.py
| +-- test_client.py
|
+-- server/
| +-- init.py
| +-- server.py
|
+-- llm/
| +-- init.py
| +-- provider.py
| +-- test_groq.py
|
+-- web/
| +-- app.py
| +-- templates/
| +-- index.html
| +-- static/
| +-- style.css
| +-- app.js
|
+-- .env
+-- .gitignore
+-- requirements.txt
+-- README.md
```

4.1 Directory Responsibilities

`server/` Contains the MCP server. The server exposes business intelligence functions as MCP tools.

`client/` Contains the MCP client. The client connects to the MCP server over Streamable HTTP, initializes an MCP session, discovers available tools, and invokes tools.

`agent/` Contains the AI agent. It connects the LLM to the MCP client and implements the tool-calling loop.

`llm/` Contains the abstraction for LLM providers. Groq is the initial provider, with the architecture designed to support OpenAI and Gemini.

`web/` Contains the FastAPI application and frontend interface.

5 High-Level Architecture

Figure 1 illustrates the overall architecture.

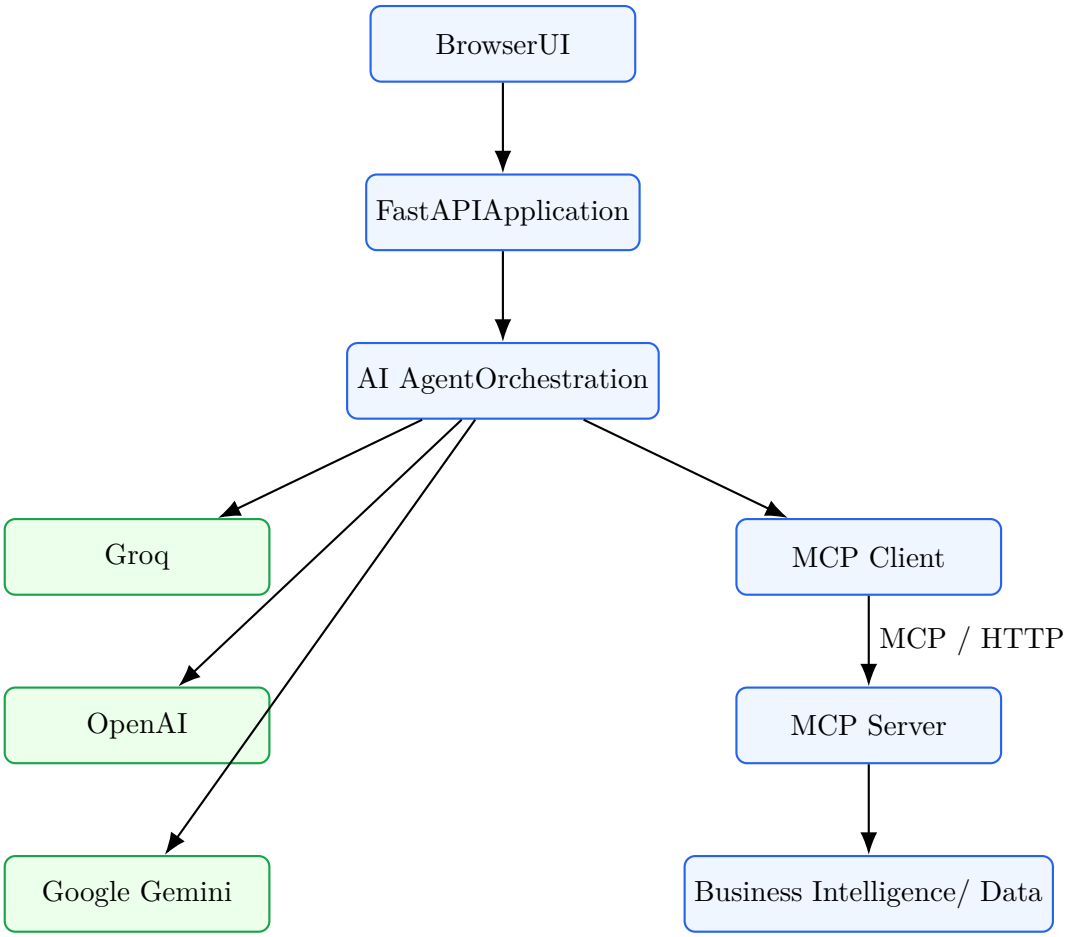


Figure 1: Overall system architecture

6 End-to-End Request Flow

The following sequence describes what happens when a user asks a business question.

1. The user enters a natural-language question into the web interface.
2. FastAPI receives the HTTP request.
3. The request is forwarded to the AI agent.
4. The agent provides the LLM with the available MCP tool definitions.
5. The LLM determines whether a tool is required.
6. If a tool is required, the LLM generates a tool call.
7. The AI agent sends that tool request to the MCP client.
8. The MCP client establishes or uses an MCP session with the MCP server.
9. The MCP server executes the requested business intelligence operation.
10. The server returns the tool result to the MCP client.
11. The result is supplied back to the LLM.
12. The LLM generates the final natural-language answer.
13. FastAPI returns the answer to the browser.

The logical flow is therefore:

User → FastAPI → AI Agent → LLM → MCP Client → MCP Server → Business Tool

The result then travels in the reverse direction.

7 Understanding Model Context Protocol

7.1 What is MCP?

Model Context Protocol is a standardized protocol for connecting AI applications to external tools, data sources, and capabilities.

Instead of implementing custom integrations for every LLM and every external system, MCP provides a standardized interface through which an AI application can discover and invoke capabilities.

Conceptually:

LLM → MCP Client → MCP Server → Tool/Data

7.2 Why MCP is useful

Without a standardized protocol, an AI application may need custom integration logic for each external service.

MCP separates these responsibilities.

The LLM is responsible for reasoning.

The MCP client is responsible for communication.

The MCP server is responsible for exposing controlled capabilities.

The underlying tools are responsible for performing actual operations.

8 MCP Server

The MCP server is the provider of business capabilities.

Typical tools can include:

- `get`$_{s}ales_{d}ata$
- `calculate`$_{g}rowth$
- `get`$_{b}usiness_{c}ontext$

A simplified conceptual implementation is:

```
@mcp.tool()
def get_sales_data():
"""
Return business sales information.
"""
return sales_data
```

The important point is that these functions are exposed as MCP tools rather than being called directly by the LLM.

9 MCP Client

The MCP client is responsible for connecting to the MCP server.

The client uses Streamable HTTP.

The simplified connection architecture is:

$$\text{MCP Client} \xrightarrow{\text{HTTP}} \text{MCP Server}$$

The client initializes a session and discovers tools.

Conceptually:

```
async with streamable_http_client(
MCP_SERVER_URL
) as (read_stream, write_stream):

async with ClientSession(
    read_stream,
    write_stream
) as session:

    await session.initialize()

    result = await session.list_tools()
```

The exact field names depend on the installed MCP Python SDK version.

For the SDK used in this project, MCP tool schemas are exposed using Python-style names such as:

```
tool.input_schema
```

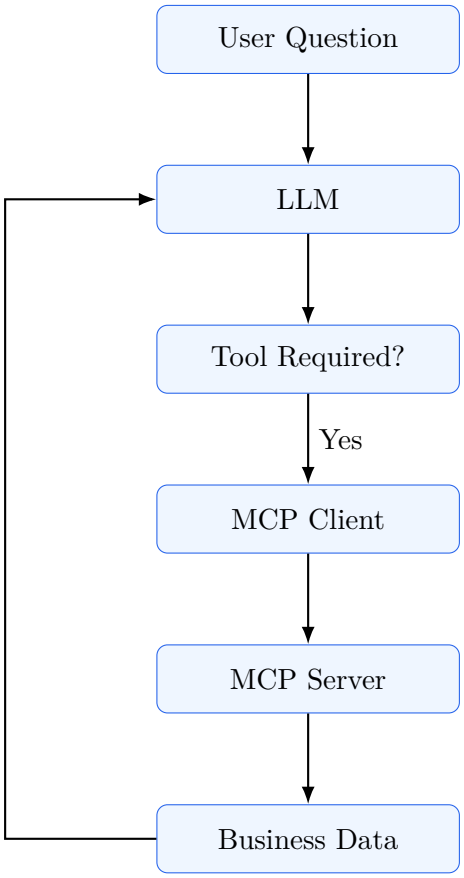
10 AI Agent

The AI agent is the orchestration layer between the LLM and MCP.

Its responsibilities include:

1. Discovering MCP tools.
2. Translating MCP tool schemas into LLM-compatible tool schemas.
3. Sending available tools to the LLM.
4. Receiving tool calls generated by the LLM.
5. Executing the corresponding MCP tools.
6. Returning MCP results to the LLM.
7. Producing the final answer.

The core loop is:



11 Groq Integration

Groq is used as one of the supported LLM providers.

The API key is stored in the environment rather than inside the source code.

The configuration is:

```
GROQ_API_KEY=your_groq_api_key
GROQ_MODEL=your_available_groq_model
LLM_PROVIDER=groq
```

The model should be selected from the models available to the specific Groq API project.

The application should not assume that every model visible in public documentation is necessarily available to every API key.

12 Multi-LLM Architecture

A major extension of the project is support for multiple LLM providers.

The conceptual architecture is:

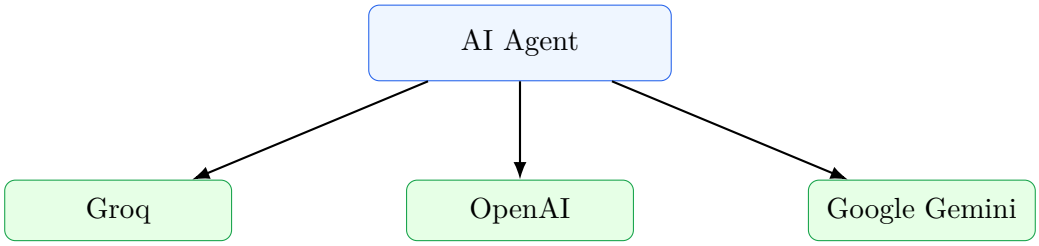


Figure 2: Multi-provider LLM architecture

This separation is important because the MCP server should remain independent of the selected LLM.

Therefore:

MCP infrastructure is LLM-provider independent.

A user can potentially change the LLM provider without rebuilding the MCP server.

13 Environment Configuration

Create a file named:

```
.env
```

Example configuration:

```
MCP_SERVER_URL=http://127.0.0.1:8000/mcp

LLM_PROVIDER=groq

GROQ_API_KEY=your_groq_api_key
GROQ_MODEL=your_available_groq_model

OPENAI_API_KEY=your_openai_api_key

GEMINI_API_KEY=your_gemini_api_key
```

Never commit this file to Git.

The `.gitignore` file should include:

```
.env
.venv/
pycache/
*.pyc
```

14 Installation

14.1 Prerequisites

The following software is required:

- Python 3.11 or newer
- pip
- Git
- A Groq API key for Groq testing
- Optional OpenAI API key
- Optional Gemini API key

Python 3.13 may also be used if all project dependencies support it.

14.2 Clone the Repository

Clone the project:

```
git clone <YOUR_REPOSITORY_URL>
cd mcp-business-intelligence
```

14.3 Create a Virtual Environment

macOS/Linux:

```
python3 -m venv .venv
source .venv/bin/activate
```

Windows PowerShell:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

14.4 Install Dependencies

If `requirements.txt` exists:

```
pip install -r requirements.txt
```

Otherwise install the primary packages:

```
pip install fastapi uvicorn python-dotenv groq mcp
```

15 Configuration

Create the environment file:

```
touch .env
```

Add:

```
MCP_SERVER_URL=http://127.0.0.1:8000/mcp

GROQ_API_KEY=YOUR_KEY
GROQ_MODEL=YOUR_AVAILABLE_MODEL

LLM_PROVIDER=groq
```

The API key must never be included in source code.

16 Running the Application

The application is easiest to run using multiple terminals.

16.1 Terminal 1: MCP Server

Activate the virtual environment and run:

```
python server/server.py
```

Expected output:

```
Uvicorn running on http://127.0.0.1:8000
```

The MCP server should remain running.

16.2 Terminal 2: MCP Client Test

From the repository root:

```
python -m client.test_client
```

This verifies:

- MCP connection
- Session initialization
- Tool discovery
- Tool invocation

16.3 Terminal 3: AI Agent Test

Run:

```
python -m agent.test_agent
```

The expected flow is:

```
Connecting to MCP server...

Calling MCP tool: get_sales_data

==============================
AI ANSWER
<LLM-generated business analysis>
==============================
MCP TOOLS USED

get_sales_data
```

16.4 Terminal 4: Web Application

Run:

```
uvicorn web.app:app --reload --port 8080
```

Then open:

http://127.0.0.1:8080

17 Testing Strategy

Testing is performed incrementally.

17.1 Test 1: MCP Server

Verify that the server starts:

```
python server/server.py
```

Expected:

```
Uvicorn running on http://127.0.0.1:8000
```

17.2 Test 2: MCP Client

Run:

```
python -m client.test_client
```

The client should discover the server's tools.

17.3 Test 3: Groq

Run the dedicated Groq test:

```
python llm/test_groq.py
```

This verifies the API key and model independently of MCP.

17.4 Test 4: AI Agent

Run:

```
python -m agent.test_agent
```

This tests the complete:

$$\text{LLM} \rightarrow \text{MCP Tool} \rightarrow \text{MCP Result} \rightarrow \text{LLM}$$

pipeline.

17.5 Test 5: Web Application

Start:

```
uvicorn web.app:app --reload --port 8080
```

Open the application in a browser and submit a business question.

18 Troubleshooting

18.1 404 at Root

If the server shows:

```
GET / HTTP/1.1 404 Not Found
```

the application does not currently have a route for `/`.
If a web interface is intended, verify that `web/app.py` contains a root route.

18.2 Jinja2 Template Error

An error such as:

```
TypeError: unhashable type: 'dict'
```

can occur when `TemplateResponse` is called with arguments that do not match the installed Starlette/FastAPI API.
The route should be written according to the installed FastAPI/Starlette version.

18.3 MCP Import Error

If Python reports:

```
ModuleNotFoundError:
No module named 'client.client'
```

ensure that:

```
client/init.py
```

exists and execute the test from the repository root:

```
python -m client.test_client
```

18.4 MCP Streamable HTTP Error

If the MCP SDK reports:

```
ValueError:
not enough values to unpack
(expected 3, got 2)
```

the installed MCP SDK returns two stream objects.
The connection should therefore use:

```
async with streamable_http_client(
MCP_SERVER_URL
) as (read_stream, write_stream):
...
```

18.5 MCP inputSchema Error

If the SDK reports:

```
AttributeError:
'Tool' object has no attribute 'inputSchema'
```

use:

```
tool.input_schema
```

rather than:

```
tool.inputSchema
```

18.6 Groq Model Not Found

If Groq returns:

```
model_not_found
```

the configured model may not be available to the current API project.

The application should retrieve or inspect the models available to the specific API key and set:

```
GROQ_MODEL=<available-model>
```

Do not assume that a model identifier from another project or account is available to the current API key.

19 Security

API keys are secrets and must not be committed to Git.

The following files must never contain real credentials:

- Python source files
- HTML files
- JavaScript files
- README files
- LaTeX documentation
- Git history

Use environment variables instead.

The application should also avoid exposing API keys through frontend JavaScript.

The browser should communicate with the backend, while the backend uses the private API keys.

Correct:

$$\text{Browser} \rightarrow \text{FastAPI} \rightarrow \text{LLM API}$$

Incorrect:

$$\text{Browser} \rightarrow \text{LLM API using embedded private API key}$$

20 Deployment Architecture

For deployment, the local architecture becomes a public service.

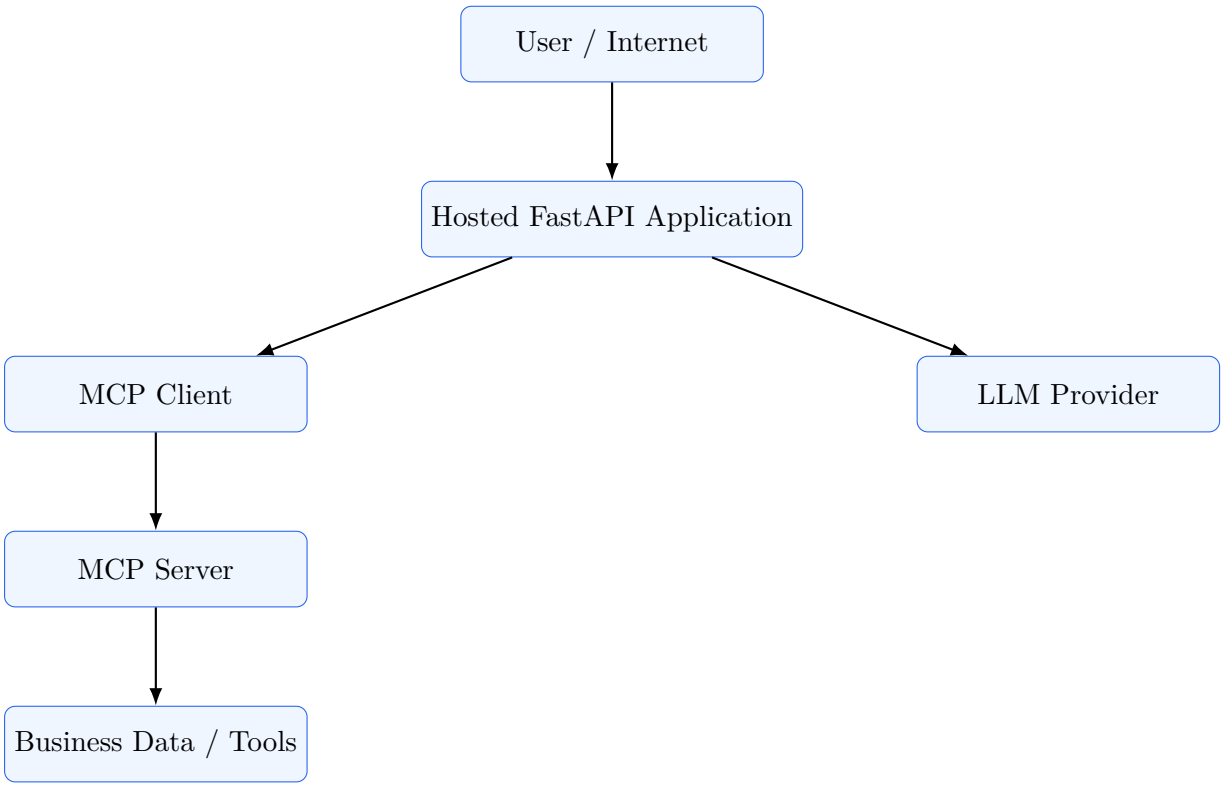


Figure 3: Deployment architecture

The deployed application should use HTTPS.

Environment variables should be configured through the hosting provider's secret/environment-variable system.

21 Production Considerations

Before production deployment, the following should be considered:

- HTTPS
- Authentication
- Rate limiting
- Request validation
- API key protection
- Logging
- Error handling
- MCP server authorization
- Tool-level access control

- Input sanitization
- Monitoring

The MCP server should expose only the operations that the AI agent is allowed to perform.
For example, a read-only business intelligence server is safer than a server that allows arbitrary database modification.

22 Business Value

The application demonstrates how MCP can automate business intelligence tasks.
Traditional workflow:

1. Analyst receives a business question.
2. Analyst opens a reporting system.
3. Analyst searches for relevant information.
4. Analyst exports data.
5. Analyst calculates metrics.
6. Analyst writes a report.

AI + MCP workflow:

1. User asks a natural-language question.
2. LLM identifies the required capability.
3. MCP client invokes the appropriate tool.
4. MCP server retrieves or calculates the information.
5. LLM summarizes the result.

This can significantly reduce repetitive analytical work.

23 Potential Industry Applications

The architecture can be generalized beyond business intelligence.

23.1 Finance

MCP tools could provide:

- Portfolio information
- Financial reports
- Risk metrics
- Transaction analysis

23.2 Healthcare

Subject to appropriate privacy and compliance controls, MCP could expose:

- Clinical data retrieval
- Scheduling tools
- Research databases
- Operational analytics

23.3 Retail

Potential tools include:

- Inventory lookup
- Sales analysis
- Customer segmentation
- Product performance

23.4 Supply Chain

Possible capabilities include:

- Inventory status
- Shipment tracking
- Supplier information
- Delivery performance

23.5 Customer Support

MCP could connect an AI assistant to:

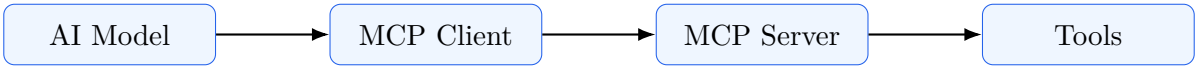
- Customer records
- Ticketing systems
- Product documentation
- Order information

24 Why MCP Matters

The key advantage is separation of concerns.

Without MCP, an AI application may contain many provider-specific integrations.

With MCP:



Each layer can evolve independently.

A new LLM can be introduced without rebuilding the business tool layer.

A new business tool can be introduced without redesigning the entire LLM application.

This modularity is one of the most important architectural benefits of MCP.

25 Demonstration Plan

A five-minute demonstration can follow the structure below.

25.1 Minute 0–1: Introduce the Problem

Explain:

> "This application is an AI-powered business intelligence assistant. Instead of manually searching business data, a user can ask questions in natural language."

25.2 Minute 1–2: Explain MCP

Show the architecture:

$$\text{LLM} \rightarrow \text{MCP Client} \rightarrow \text{MCP Server} \rightarrow \text{Tools}$$

Explain that the MCP server exposes controlled business capabilities.

25.3 Minute 2–3: Demonstrate Tool Discovery

Show the MCP client.

Run:

```
python -m client.test_client
```

Explain that the client connects to the MCP server and discovers available tools.

25.4 Minute 3–4: Demonstrate AI Tool Calling

Run:

```
python -m agent.test_agent
```

Ask a question such as:

> "Which product is underperforming?"

Show that the LLM chooses an MCP tool.

25.5 Minute 4–5: Demonstrate the Web Application

Open the deployed web application.

Ask a business question and show:

- The user question
- The selected LLM provider
- MCP tool execution
- The final business analysis

Conclude by explaining that the same MCP infrastructure can support multiple LLM providers.

26 Future Enhancements

Potential improvements include:

- RAG for large business documents
- Vector database integration
- Authentication
- Persistent conversation history
- Streaming responses
- Advanced business dashboards
- Multiple MCP servers
- Database-backed business tools
- Tool permission management
- Usage and cost tracking
- OpenAI provider
- Google Gemini provider
- Automatic provider fallback

A particularly useful extension is RAG.

Instead of sending an entire knowledge base to the LLM, the system can retrieve only the most relevant documents.

The resulting architecture becomes:

$$\text{User Question} \rightarrow \text{Retriever} \rightarrow \text{Relevant Context} \rightarrow \text{LLM}$$

This can reduce token consumption and improve response quality.

27 Conclusion

This project demonstrates a complete AI application built around the Model Context Protocol.

The architecture separates:

- User interaction
- AI reasoning
- LLM provider
- MCP communication
- Business tools

The most important architectural property is that the LLM does not need to directly understand how every business system works.

Instead, the MCP server exposes standardized tools.

The AI agent discovers these tools and determines when they should be used.

The final system can therefore be summarized as:

Natural Language → LLM Reasoning → MCP Tool → Business Data → AI Answer

This architecture provides a practical foundation for building AI-powered enterprise applications where language models need secure, controlled, and reusable access to external capabilities.